uart_tx.sv: что было, что исправлено и почему

Папка `C:\FPGA_PROJECT\Uart` · 1 октября 2026 · Questa Altera Starter FPGA Edition 2025.2

> **Коротко.** Вы переписали передатчик на схему из двух блоков: `always_comb` решает, каким всё станет на следующем такте, а `always_ff` на фронте `clk` это запоминает. Идея правильная, я её сохранил и все ваши комментарии тоже. Исправлено всё, из-за чего vlog выдавал 10 ошибок, плюс логические ошибки, которые компилятор не видит (автомат застревал в START, регистры меняли два блока сразу). В тестбенче переименован сброс под ваш новый порт `rst`. После этого тест в Questa прошёл: **5 из 5 байт OK, TEST PASSED, 0 ошибок, 0 предупреждений**.
>
> Полные копии файлов до правки лежат в `backup\uart_tx_before_fix.sv` и `backup\tb_uart_tx_before_fix.sv`.

Подсветка строк: ошибка в старом коде | изменено в новом коде | только текст сообщения

1. Старый код (как был до правки)

uart_tx.sv — старая версия

```
1  // UART-передатчик: 8 бит данных, без чётности, 1 стоп-бит (8N1).
2  // Старт передачи: импульс tx_start на 1 такт при busy == 0.
3  module uart_tx #(
4      parameter int CLK_HZ = 50_000_000,  //тактовая частота платы
5      parameter int BAUD   = 115_200      // скорость передаччи бит/сек
6  ) (
7      input  logic       clk,      // тактовый импульс как обысно
8      input  logic       rst,     // импульс сброса
9      input  logic       tx_start,  // запрос на отправку байта
10     input  logic [7:0] tx_data,   // байт который нуно передать по Uart
11     output logic       tx,        // линия TX (в покое = 1)
12     output logic       busy       // 1, пока идёт передача
13 );
14     localparam int CLKS_PER_BIT = CLK_HZ / BAUD;  // 434 при 50 МГц / 115200 количесвто тактов на один бит
   информации
15     localparam int CNT_W        = $clog2(CLKS_PER_BIT); // количсетво бит для счетчика
16
17     typedef enum logic [1:0] {
18      IDLE, // стоим ждем
19      START, // начинаем сроку байт
20      DATA, // передаем данные
21      STOP // стоп-бит заверили передачу байт
22      }
23      state_t;
24     state_t state, state_next; // настоящее и последущее состояние передатчика
25
26
27 // опишем из чего состоит каждое состояние
28     always_comb begin : States
29         state_next = state; // если ничего не меняемся то стоим на месте просто
30         case(state)
31            IDLE: begin
32               clk_cnt <= '0; // держим на нуле наш сетчик тактов clk на один бит
33               tx      <= 1'b1; // держим линию на высоком напряжении (логическая 1)
34               if (tx_start) begin
35                   shreg         <= tx_data; // вот пришел наш байт на отправку мы засунули его в шрег
```

```
                tx_start      <= 1'b0; // переводим в старт наш стартовый бит (кидаем в маленькое напряжение)
                state_next    <= START; // переводим состояние в стартовое
             end
           end
          START: begin
             if (bit_done): begin // когда счетчикотсчитал 434 такта нужные на наш бит данных
                bit_idx <= 3'b0; // первый пришедший бит его индекс нулевой записан 3 битами
                tx      <= shreg[0] // первым битом вперед начинаем запись
                state_next <= state;
               end
           end
          DATA: begin
            if (bit_done): begin
                if (bit_idx == 3'b7): begin
                    tx         <= 1'b1; // присваиваем на линию передачи стоп-бит
                    state_next <= STOP; // следующее состояние будет стоповое логично
                end
                else: begin
                    bit_idx <= bit_idx + 3'b1; // увеличиваем номер бита следующего
                    tx      <= shreg[bit_idx + 1'b1]; // вносим в линию передачи бит
                end
             end
          end
          STOP: begin
            if (bit_done) state_net <= IDLE;
          end
          default : state_next <= IDLE; // если вдруг ошибка то линия выйдет на ожидание тут же
        endcase
    end
    logic [CNT_W-1:0] clk_cnt;   // счётчик тактов внутри одного бита
    logic [2:0]       bit_idx;   // номер передаваемого бита данных
    logic [7:0]       shreg;     // защёлкнутый байт то есть когда прихоит наш байт который надо отправить по протоколу то он ложится сюда извне

    wire bit_done = (clk_cnt == CNT_W'(CLKS_PER_BIT - 1)); // равен единице когда счетчик clk_cnt дошел и отсчитал 434 такта (столько нужно на один бит информации)

    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            state   <= IDLE;
            clk_cnt <= '0;
            bit_idx <= '0;
            shreg   <= '0;
            tx      <= 1'b1;
        end else begin
            clk_cnt <= bit_done ? '0 : clk_cnt + 1'b1;
            state   <= state_next;
        end
    end

    assign busy = (state != IDLE);
endmodule
```

tb_uart_tx.sv — старая версия

```
// Тестбенч: отправляет несколько байт и декодирует линию TX обратно,
// проверяя старт-бит, данные и стоп-бит на середине каждого бита.
`timescale 1ns/1ps
module tb_uart_tx;
    localparam int CLK_HZ = 50_000_000;
    localparam int BAUD   = 115_200;
    localparam int CLKS_PER_BIT = CLK_HZ / BAUD;
    localparam real CLK_NS = 20.0;
```

```
    logic clk = 0, rst_n = 0, tx_start = 0;
    logic [7:0] tx_data = 0;
    logic tx, busy;
    int errors = 0;

    uart_tx #(.CLK_HZ(CLK_HZ), .BAUD(BAUD)) dut (.*);

    always #(CLK_NS/2) clk = ~clk;

    // Приёмник-эталон: ждёт спада, затем сэмплирует середины битов
    task automatic check_byte(input logic [7:0] expected);
        logic [7:0] got;
        @(negedge tx);
        repeat (CLKS_PER_BIT/2) @(posedge clk);
        if (tx !== 1'b0) begin $display("ОШИБКА: нет старт-бита"); errors++; end
        for (int i = 0; i < 8; i++) begin
            repeat (CLKS_PER_BIT) @(posedge clk);
            got[i] = tx;
        end
        repeat (CLKS_PER_BIT) @(posedge clk);
        if (tx !== 1'b1) begin $display("ОШИБКА: нет стоп-бита"); errors++; end
        if (got !== expected) begin
            $display("ОШИБКА: ожидали 0x%02h, получили 0x%02h", expected, got); errors++;
        end else
            $display("OK: 0x%02h ('%c')", got, got);
    endtask

    task automatic send(input logic [7:0] b);
        @(posedge clk);
        while (busy) @(posedge clk);
        tx_data <= b; tx_start <= 1;
        @(posedge clk); tx_start <= 0;
        @(posedge clk);  // даём busy подняться
    endtask

    logic [7:0] seq [5];
    initial begin
        seq[0] = 8'h55; seq[1] = 8'h00; seq[2] = 8'hFF; seq[3] = 8'h41; seq[4] = 8'hA5;
    end

    initial begin
        repeat (5) @(posedge clk);
        rst_n = 1;
        repeat (5) @(posedge clk);
        if (tx !== 1'b1) begin $display("ОШИБКА: в покое TX != 1"); errors++; end
        fork
            for (int i = 0; i < 5; i++) send(seq[i]);
            for (int j = 0; j < 5; j++) check_byte(seq[j]);
        join
        wait (!busy);
        if (errors == 0) $display("ТЕСТ ПРОЙДЕН");
        else             $display("ТЕСТ НЕ ПРОЙДЕН: %0d ошибок", errors);
        $finish;
    end

    // Сторож: не даёт симуляции зависнуть
    initial begin
        #(CLK_NS * CLKS_PER_BIT * 10 * 20);
        $display("ОШИБКА: таймаут симуляции");
        $finish;
    end

    initial begin
```

```
        if ($test$plusargs("vcd")) begin
            $dumpfile("tb_uart_tx.vcd");
            $dumpvars(0, tb_uart_tx);
        end
    end
endmodule
```

2. Новый код (после исправления)

uart_tx.sv — новая версия

```
// UART-передатчик: 8 бит данных, без чётности, 1 стоп-бит (8N1).
// Старт передачи: импульс tx_start на 1 такт при busy == 0.
module uart_tx #(
    parameter int CLK_HZ = 50_000_000,  //тактовая частота платы
    parameter int BAUD   = 115_200      // скорость передаччи бит/сек
) (
    input  logic       clk,      // тактовый импульс как обысно
    input  logic       rst,     // импульс сброса
    input  logic       tx_start,  // запрос на отправку байта
    input  logic [7:0] tx_data,   // байт который нуно передать по Uart
    output logic       tx,        // линия TX (в покое = 1)
    output logic       busy       // 1, пока идёт передача
);
    localparam int CLKS_PER_BIT = CLK_HZ / BAUD;  // 434 при 50 МГц / 115200 количесвто тактов на один бит информации
    localparam int CNT_W        = $clog2(CLKS_PER_BIT); // количсетво бит для счетчика

    typedef enum logic [1:0] {
     IDLE, // стоим ждем
     START, // начинаем сроку байт
     DATA, // передаем данные
     STOP // стоп-бит заверили передачу байт
     }
     state_t;
    state_t state, state_next; // настоящее и последущее состояние передатчика

    // Регистры (меняются только в always_ff) и их "следующие" значения (считаются в always_comb)
    logic [CNT_W-1:0] clk_cnt, clk_cnt_next;   // счётчик тактов внутри одного бита
    logic [2:0]       bit_idx, bit_idx_next;   // номер передаваемого бита данных
    logic [7:0]       shreg,   shreg_next;     // защёлкнутый байт то есть когда прихоит наш байт который надо отправить по протоколу то он ложится сюда извне
    logic             tx_next;                 // следующее значение линии TX

    wire bit_done = (clk_cnt == CNT_W'(CLKS_PER_BIT - 1)); // равен единице когда счетчик clk_cnt дошел и отсчитал 434 такта (столько нужно на один бит информации)

// опишем из чего состоит каждое состояние
    always_comb begin : States
        state_next   = state; // если ничего не меняемся то стоим на месте просто
        clk_cnt_next = bit_done ? '0 : clk_cnt + 1'b1; // по умолчанию счетчик тикает и сбрасывается в 0 на конце бита
        bit_idx_next = bit_idx; // по умолчанию все регистры хранят свое значение
        shreg_next   = shreg;
        tx_next      = tx;
        case (state)
           IDLE: begin
              clk_cnt_next = '0; // держим на нуле наш сетчик тактов clk на один бит
              tx_next      = 1'b1; // держим линию на высоком напряжении (логическая 1)
              if (tx_start) begin
                  shreg_next = tx_data; // вот пришел наш байт на отправку мы засунули его в шрег
                  tx_next    = 1'b0; // переводим в старт наш стартовый бит (кидаем в маленькое напряжение)
                  state_next = START; // переводим состояние в стартовое
              end
            end
           START: begin
              if (bit_done) begin // когда счетчикотсчитал 434 такта нужные на наш бит данных
```

```
                bit_idx_next = 3'd0; // первый пришедший бит его индекс нулевой записан 3 битами
                tx_next      = shreg[0]; // первым битом вперед начинаем запись
                state_next   = DATA; // старт-бит отправлен, переходим к данным
              end
            end
          DATA: begin
            if (bit_done) begin
                if (bit_idx == 3'd7) begin
                    tx_next    = 1'b1; // присваиваем на линию передачи стоп-бит
                    state_next = STOP; // следующее состояние будет стоповое логично
                end
                else begin
                    bit_idx_next = bit_idx + 3'd1; // увеличиваем номер бита следующего
                    tx_next      = shreg[bit_idx + 3'd1]; // вносим в линию передачи бит
                end
             end
          end
          STOP: begin
            if (bit_done) state_next = IDLE;
          end
          default : state_next = IDLE; // если вдруг ошибка то линия выйдет на ожидание тут же
        endcase
    end

    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            state   <= IDLE;
            clk_cnt <= '0;
            bit_idx <= '0;
            shreg   <= '0;
            tx      <= 1'b1;
        end else begin
            state   <= state_next;   // на каждом фронте clk регистры просто
            clk_cnt <= clk_cnt_next; // забирают значения, посчитанные в always_comb
            bit_idx <= bit_idx_next;
            shreg   <= shreg_next;
            tx      <= tx_next;
        end
    end

    assign busy = (state != IDLE);
endmodule
```

tb_uart_tx.sv — новая версия

```
// Тестбенч: отправляет несколько байт и декодирует линию TX обратно,
// проверяя старт-бит, данные и стоп-бит на середине каждого бита.
`timescale 1ns/1ps
module tb_uart_tx;
    localparam int CLK_HZ = 50_000_000;
    localparam int BAUD   = 115_200;
    localparam int CLKS_PER_BIT = CLK_HZ / BAUD;
    localparam real CLK_NS = 20.0;

    logic clk = 0, rst = 1, tx_start = 0;
    logic [7:0] tx_data = 0;
    logic tx, busy;
    int errors = 0;

    uart_tx #(.CLK_HZ(CLK_HZ), .BAUD(BAUD)) dut (.*);

    always #(CLK_NS/2) clk = ~clk;

    // Приёмник-эталон: ждёт спада, затем сэмплирует середины битов
```

```
    task automatic check_byte(input logic [7:0] expected);
        logic [7:0] got;
        @(negedge tx);
        repeat (CLKS_PER_BIT/2) @(posedge clk);
        if (tx !== 1'b0) begin $display("ERROR: no start bit"); errors++; end
        for (int i = 0; i < 8; i++) begin
            repeat (CLKS_PER_BIT) @(posedge clk);
            got[i] = tx;
        end
        repeat (CLKS_PER_BIT) @(posedge clk);
        if (tx !== 1'b1) begin $display("ERROR: no stop bit"); errors++; end
        if (got !== expected) begin
            $display("ERROR: expected 0x%02h, got 0x%02h", expected, got); errors++;
        end else
            $display("OK: 0x%02h ('%c')", got, got);
    endtask

    task automatic send(input logic [7:0] b);
        @(posedge clk);
        while (busy) @(posedge clk);
        tx_data <= b; tx_start <= 1;
        @(posedge clk); tx_start <= 0;
        @(posedge clk);  // даём busy подняться
    endtask

    logic [7:0] seq [5];
    initial begin
        seq[0] = 8'h55; seq[1] = 8'h00; seq[2] = 8'hFF; seq[3] = 8'h41; seq[4] = 8'hA5;
    end

    initial begin
        repeat (5) @(posedge clk);
        rst = 0;
        repeat (5) @(posedge clk);
        if (tx !== 1'b1) begin $display("ERROR: idle TX != 1"); errors++; end
        fork
            for (int i = 0; i < 5; i++) send(seq[i]);
            for (int j = 0; j < 5; j++) check_byte(seq[j]);
        join
        wait (!busy);
        if (errors == 0) $display("TEST PASSED");
        else             $display("TEST FAILED: %0d errors", errors);
        $finish;
    end

    // Сторож: не даёт симуляции зависнуть
    initial begin
        #(CLK_NS * CLKS_PER_BIT * 10 * 20);
        $display("ERROR: simulation timeout");
        $finish;
    end

    initial begin
        if ($test$plusargs("vcd")) begin
            $dumpfile("tb_uart_tx.vcd");
            $dumpvars(0, tb_uart_tx);
        end
    end
endmodule
```

3. Каждая изменённая строка: зачем и почему так правильно

> **Главная идея двухблочного автомата, на ней держатся почти все правки.**
>
> - **Регистры** (`state`, `clk_cnt`, `bit_idx`, `shreg`, `tx`) — это триггеры. Они меняются **только** в `always_ff`, только на фронте `clk`, и там пишут `<=`.
> - **Комбинационная логика** в `always_comb` только **вычисляет**, какими регистры станут на следующем такте. Результат кладётся в пары с окончанием `_next`, и там пишут `=`.
> - Одному сигналу — один блок-хозяин. Если `tx` меняют и `always_comb`, и `always_ff`, это «несколько драйверов»: в железе два выхода спорят за один провод, а Questa такое не компилирует.

uart_tx.sv

№	Было (строка старого файла)	Стало (строка нового файла)	Зачем изменил и почему так правильно

1	65–67:
	`logic [CNT_W-1:0] clk_cnt;`
	`logic [2:0] bit_idx;`
	`logic [7:0] shreg;`
	(стояли ПОСЛЕ always_comb)
		27–29:
		`logic [CNT_W-1:0] clk_cnt, clk_cnt_next;`
		`logic [2:0] bit_idx, bit_idx_next;`
		`logic [7:0] shreg, shreg_next;`
		(стоят ДО always_comb)
			Перенёс выше. Компилятор читает файл сверху вниз, и имя можно использовать только после объявления. На строке 32 он ещё не знал, что такое `clk_cnt`, отсюда ошибки *Undefined variable: 'clk_cnt'*, *'shreg'*.
			Добавил пары `_next`. У каждого регистра теперь есть «следующее значение», как у вас уже было `state` / `state_next`. Это та же идея, просто для всех регистров.

2	(не было)	30: `logic tx_next;`	`tx` — выход модуля и регистр, у него тоже должно быть «следующее значение». Сам `tx` объявлен в списке портов, поэтому тут объявлен только `tx_next`.
3	69: `wire bit_done = ...` (ниже always_comb)	32: `wire bit_done = ...` (выше always_comb)	Та же причина, что в п.1: `bit_done` используется в строке 41, а объявлен был в 69. Ошибка *Undefined variable: 'bit_done'*. Сама формула не менялась: `bit_done = 1` на последнем, 434-м такте бита.

4	(не было)	37–40:
		`clk_cnt_next = bit_done ? '0 : clk_cnt + 1'b1;`
		`bit_idx_next = bit_idx;`
		`shreg_next = shreg;`
		`tx_next = tx;`
			Значения по умолчанию в начале `always_comb`. В комбинационном блоке каждый сигнал должен получить значение при *любом* пути через `case` и `if`. Иначе синтез поставит **защёлку (latch)**, то есть память без тактового сигнала, а это источник странных ошибок на плате. Умолчание «оставь как было» значит, что регистр хранит своё значение, если состояние его не трогает. Это как ваша строка 29 `state_next = state`, только для остальных. Формулу счётчика я перенёс сюда из `always_ff` (старая строка 79), чтобы в IDLE её можно было перекрыть нулём (п.5).

5	32–33:
	`clk_cnt <= '0;`
	`tx <= 1'b1;`
		43–44:
		`clk_cnt_next = '0;`
		`tx_next = 1'b1;`
			Две причины. **(а)** В `always_comb` пишут блокирующее `=`, а неблокирующее `<=` — для `always_ff`, где значения обновляются разом на фронте. **(б)** `clk_cnt` и `tx` — регистры, их уже меняет `always_ff`. Меняем их «следующие» копии, и конфликта драйверов нет. Смысл ваш: в покое счётчик на нуле, линия в единице.

6	35: `shreg <= tx_data;`	46: `shreg_next = tx_data;`	То же, что п.5. На следующем фронте `clk` в `always_ff` выполнится `shreg <= shreg_next`, и байт защёлкнется в регистр.
7	36: `tx_start <= 1'b0;`	47: `tx_next = 1'b0;`	**Логическая ошибка.** `tx_start` — **вход** модуля, им управляет тот, кто подключил передатчик (у нас тестбенч). Модуль не может присваивать своему входу. По вашему комментарию нужно было «кинуть линию в низкое напряжение», то есть выдать **старт-бит на линию** `tx`. На следующем фронте одновременно станут `tx = 0` и `state = START`, и старт-бит начнётся.
8	37: `state_next <= START;`	48: `state_next = START;`	Только оператор: в `always_comb` пишут `=` (см. п.5а).
9	41: `if (bit_done): begin`	52: `if (bit_done) begin`	**Синтаксис.** Двоеточие в SystemVerilog ставится только после метки `case` (`IDLE:`, `default :`) и перед именем блока (`begin : States`). После `if (...)` его нет. Отсюда ошибка *near ":": syntax error, unexpected* ':'.
10	42: `bit_idx <= 3'b0;`	53: `bit_idx_next = 3'd0;`	Регистр → `_next`, `<=` → `=` (п.5). `3'b0` был записан правильно, ноль есть ноль. Я написал `3'd0` в десятичной форме только для единообразия с `3'd7` из п.14.
11	43: `tx <= shreg[0]` (нет «;»)	54: `tx_next = shreg[0];`	**Пропущена точка с запятой.** Компилятор замечает это только на **следующей** строке, поэтому ошибка была указана в строке 44: *near "state_next": ... expecting* ';'. Это частая ловушка: если ошибка непонятная, смотрите строку выше. `shreg[0]` идёт первым, потому что UART передаёт младший бит первым.
12	44: `state_next <= state;`	55: `state_next = DATA;`	**Логическая ошибка, компилятор её не видит.** `state_next = state` значит «остаться в START». Автомат вечно слал бы старт-бит, а тестбенч упал бы по таймауту. После старт-бита надо перейти к данным, в `DATA`.
13	48: `if (bit_done): begin`	59: `if (bit_done) begin`	Лишнее двоеточие, как в п.9.

14	49:
	`if (bit_idx == 3'b7):`
	`begin`
		60:
		`if (bit_idx == 3'd7)`
		`begin`
			`3'b7` — «трёхбитное **двоичное** число 7», но в двоичной записи есть только цифры 0 и 1. Ошибка *Illegal digit for specified base*. Семь пишется `3'd7` (десятичная) или `3'b111` (двоичная). Лишнее двоеточие убрано, как в п.9.

15	50–51:
	`tx <= 1'b1;`
	`state_next <= STOP;`
		61–62:
		`tx_next = 1'b1;`
		`state_next = STOP;`
			Как п.5. После 8-го бита (номер 7) на линию выдаётся стоп-бит, единица, и автомат переходит в `STOP`.

16	53: `else: begin`	64: `else begin`	Лишнее двоеточие. Из-за него парсер «сбился», и следующие сообщения (строка 55, *missing* '::') — последствия этой ошибки, а не отдельные проблемы.
17	54: `bit_idx <= bit_idx + 3'b1;`	65: `bit_idx_next = bit_idx + 3'd1;`	Как п.5 и п.10. Справа стоит текущий номер бита из регистра, слева — номер на следующий такт.
18	55: `tx <= shreg[bit_idx + 1'b1];`	66: `tx_next = shreg[bit_idx + 3'd1];`	Как п.5. **Почему `+1` правильно:** в этот момент `bit_idx` ещё указывает на бит, который сейчас на линии, а выдать надо следующий. Индекс `bit_idx + 1` не выходит за пределы: при `bit_idx = 7` работает другая ветка, со стоп-битом.
19	60: `if (bit_done) state_net <= IDLE;`	71: `if (bit_done) state_next = IDLE;`	**Опечатка:** `state_net` нигде не объявлен. После стоп-бита автомат возвращается в покой.
20	62: `default : state_next <= IDLE;`	73: `default : state_next = IDLE;`	Только оператор `=`. Двоеточие здесь правильное, это метка `case`.

21	79–80:
	`clk_cnt <= bit_done ? '0 : clk_cnt + 1'b1;`
	`state <= state_next;`
		85–89:
		`state <= state_next;`
		`clk_cnt <= clk_cnt_next;`
		`bit_idx <= bit_idx_next;`
		`shreg <= shreg_next;`
		`tx <= tx_next;`
			Теперь `always_ff` только запоминает. На каждом фронте `clk` все пять регистров забирают значения, посчитанные в `always_comb`. Раньше тут обновлялись только `clk_cnt` и `state`, а `tx`, `shreg` и `bit_idx` пытались менять из комбинационного блока. Ветку сброса (78–83) я не трогал: при `rst = 1` регистры получают начальные значения, `tx = 1`, то есть линия в покое.

—	8, 71–72:
	`input logic rst`
	`posedge rst / if (rst)`
		без изменений	Ваш новый сброс `rst`, активный **единицей**, оставил как есть. `posedge rst` и `if (rst)` согласованы между собой, это правильно. Строку 30 (`case(state)`) вы уже исправили сами, я только добавил пробел.

tb_uart_tx.sv (тестбенч)

№	Было	Стало	Зачем изменил и почему так правильно
22	10: `logic clk = 0, rst_n = 0, tx_start = 0;`	10: `logic clk = 0, rst = 1, tx_start = 0;`	Передатчик подключается строкой 15 `uart_tx ... dut (.*);`. `.*` значит «соедини каждый порт с сигналом **такого же имени**». У модуля порт теперь `rst`, а в тестбенче был только `rst_n`. Сигнала `rst` не нашлось, отсюда *Undefined variable:* '.*'. Переименовал. Начальное значение **1**, потому что ваш сброс активен единицей: в начале симуляции передатчик держится в сбросе.
23	52: `rst_n = 1;`	52: `rst = 0;`	Отпускаем сброс. Для `rst_n` «отпустить» значило поставить 1, для `rst` — поставить 0.

24	`24, 30, 32, 54, 60, 61, 68:`
	`$display("ОШИБКА: ...")`
	`$display("ТЕСТ ПРОЙДЕН")`
		`$display("ERROR: ...")`
		`$display("TEST PASSED")`
			В PowerShell русский текст из Questa превращался в `Đ¢ĐĐ¡Đ¢ Đ ĐĐĐĐ`: это «ТЕСТ ПРОЙДЕН», прочитанный не в той кодировке. Латиница читается в любой консоли. Комментарии в коде остались русскими, в консоль они не выводятся. Сама логика проверок не менялась.

4. Почему вчера «тест прошёл» на сломанном коде и новый файл run_tx.do

> Вчерашний прогон показал `OK` и «ТЕСТ ПРОЙДЕН», хотя `vlog` упал с 10 ошибками. Строка лога `Loading existing optimized design _opt` объясняет это: Questa взяла из папки `work` **прошлую удачную сборку**, то есть старый рабочий передатчик, и прогнала её. Ваш новый код в симуляцию вообще не попал.

Поправка к моему вчерашнему совету. Я предлагал вписать `onerror {quit -f -code 1}` прямо в строку `-do "..."`. Проверил: так Questa пишет *Warning: onerror command for use within macro* и команду игнорирует. `onerror` работает только внутри **.do-файла**, поэтому я сделал файл `run_tx.do`:

```
# Compile and run the uart_tx test.  Usage:  vsim -batch -do run_tx.do
# onerror: if vlog fails, stop here instead of simulating an old build from work/
onerror {quit -f -code 1}
if {[file exists work]} {vdel -all -lib work}
vlib work
vlog -sv uart_tx.sv tb_uart_tx.sv
vsim work.tb_uart_tx
run -all
quit -f
```

- **Строка 3, `onerror`:** если любая команда ниже упадёт (например, `vlog`), Questa сразу выйдет с кодом 1 и **не запустит** симуляцию.
- **Строка 4, `vdel -all -lib work`:** удаляет старую скомпилированную библиотеку. Старый `_opt` больше неоткуда взять.
- **Строки 5–6:** создаём чистую `work` и компилируем оба файла.
- **Строки 7–9:** загружаем тестбенч, гоняем до `$finish`, выходим.

Запуск из PowerShell в папке `C:\FPGA_PROJECT\Uart`:

```
vsim -batch -do run_tx.do
```

`-batch` вместо `-c`: у меня вариант с `-c` завис на загрузке симуляции (похоже на ту же проблему с сокетом *RPC_Init*, которую создаёт антивирус 360), а `-batch` работает без интерактивной консоли и отработал за 2 секунды. Проверил, что защита срабатывает: подложил в копию папки ваш старый сломанный файл, и скрипт остановился сразу после `vlog` с сообщением `Executing ONERROR command at macro ./run.do line 4`, симуляция не запускалась.

5. Как я понял по тесту, что всё прошло верно

5.1. Что делает тестбенч

Тестбенч играет роль **приёмника-эталона**: сам слушает линию `tx` и расшифровывает, что на ней было. Задача `check_byte` (строки 20–35):

1. `@(negedge tx)` — ждёт спада линии из 1 в 0. Это начало старт-бита.
2. `repeat (CLKS_PER_BIT/2)` — ждёт 217 тактов, то есть **середину** старт-бита, и проверяет, что там 0. Иначе *ERROR: no start bit*.
3. 8 раз ждёт ровно 434 такта и попадает в середину каждого бита данных. Записывает бит в `got[0]` … `got[7]`, младший первым.
4. Ещё 434 такта, середина стоп-бита: там должна быть 1. Иначе *ERROR: no stop bit*.
5. Сравнивает собранный байт с ожидаемым. Совпал — `OK: 0x..`, нет — *ERROR: expected …, got …* и `errors++`.

Читать в середине бита — ровно так работает настоящий приёмник UART: там сигнал уже устоялся и далеко от фронтов. Одновременно (`fork … join`) задача `send` подаёт байты в передатчик и перед каждым ждёт `busy = 0`. Так заодно проверяется, что `busy` работает.

Почему именно эти байты:

- `0x55` = 01010101 — каждый бит меняется, видна любая ошибка в длительности бита.
- `0x00` — все нули: байт отличается от «линии в нуле» только стоп-битом.
- `0xFF` — все единицы: отличается от покоя только старт-битом.
- `0x41` = 01000001 (буква 'A') — несимметричный байт. Если перепутать порядок битов, получится `0x82`, и тест это поймает.
- `0xA5` = 10100101 — ещё один смешанный байт.

5.2. Что я увидел в выводе, строка за строкой

Строка в выводе	Что она значит

`-- Compiling module uart_tx`
`-- Compiling module tb_uart_tx`
`Errors: 0, Warnings: 0`
	Оба файла скомпилировались **без ошибок и без предупреждений**. Вчера здесь было `Errors: 10` и `vlog failed`.

`Design is being optimized...`	Questa собрала дизайн **заново из нового кода**. Вчера было `Loading existing optimized design _opt`, то есть старая сборка. К тому же папку `work` я перед запуском удалил.

`OK: 0x55 ('U')`
`OK: 0x00 ('')`
`OK: 0xff ('')`
`OK: 0x41 ('A')`
`OK: 0xa5 ('')`
	Все 5 байт приняты правильно, 5 из 5. Значки `` и пустые кавычки — не ошибка: `%c` печатает байт как символ, а у `0x00`, `0xFF` и `0xA5` нет печатного символа. Важно шестнадцатеричное значение.

`TEST PASSED`	Печатается только если счётчик `errors` остался равен 0 (строка 60). Значит, не было ни одной ошибки старт-бита, стоп-бита, данных и покоя линии.

`$finish : tb_uart_tx.sv(62)`	Симуляция закончилась в **строке 62**, штатном конце теста. Если бы передатчик завис, она закончилась бы в строке 69, у «сторожа», с *ERROR: simulation timeout*.
`Time: 434390 ns`	Проверка арифметикой: 1 байт = 10 бит (старт + 8 данных + стоп) × 434 такта × 20 нс = 86 800 нс. 5 байт = 434 000 нс, плюс 390 нс на сброс и первые такты. Ровно столько же длился прогон старой рабочей версии 30 сентября в 14:22. Значит, новый код **по времени совпадает с прежним такт в такт**.
`Errors: 0, Warnings: 0 (vsim)`	Симулятор тоже не выдал ни ошибок, ни предупреждений.

5.3. Проверка, что тест вообще умеет ловить ошибки

Тест, который проходит всегда, ничего не доказывает. Поэтому я взял **копию** файлов во временной папке (ваши файлы не трогал), нарочно испортил одну строку `bit_idx == 3'd7` → `3'd6`, то есть слал бы 7 бит вместо 8, и прогнал тот же тест. Результат: *ERROR: no stop bit*, *ERROR: expected 0x55, got 0xd5* и так далее, в конце *ERROR: simulation timeout*, строки `TEST PASSED` нет. Тест ошибку поймал, значит его «пройдено» чего-то стоит. Лог ниже.

Приложение А. Полный лог удачного прогона (vsim -batch -do run_tx.do)

```
���# ** Note: (vsim-220) 'C:/altera_lite/25.1std/questa_fse/win64/../modelsim.ini' is used as the ini file.
#
# do run_tx.do
# Questa Altera Starter FPGA Edition-64 vlog 2025.2 Compiler 2025.05 May 31 2025
# Start time: 01:27:36 on Oct 01,2026
# vlog -sv uart_tx.sv tb_uart_tx.sv
# ** Note: (vlog-220) 'C:/altera_lite/25.1std/questa_fse/win64/../modelsim.ini' is used as the ini file.
# -- Compiling module uart_tx
# -- Compiling module tb_uart_tx
#
# Top level modules:
#       tb_uart_tx
# End time: 01:27:36 on Oct 01,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
# vsim work.tb_uart_tx
# Start time: 01:27:35 on Oct 01,2026
# ** Note: (vsim-3812) Design is being optimized...
# ** Note: (vopt-220) 'C:/altera_lite/25.1std/questa_fse/win64/../modelsim.ini' is used as the ini file.
# Loading sv_std.std
# Loading work.tb_uart_tx(fast)
# OK: 0x55 ('U')
# OK: 0x00 ('')
# OK: 0xff ('���')
# OK: 0x41 ('A')
# OK: 0xa5 ('���')
# TEST PASSED
# ** Note: $finish    : tb_uart_tx.sv(62)
#    Time: 434390 ns  Iteration: 2  Instance: /tb_uart_tx
# End time: 01:27:37 on Oct 01,2026, Elapsed time: 0:00:02
# Errors: 0, Warnings: 0
```

Приложение Б. Лог прогона нарочно испорченной копии (3'd7 → 3'd6)

```
���# ** Note: (vsim-220) 'C:/altera_lite/25.1std/questa_fse/win64/../modelsim.ini' is used as the ini file.
#
# do run.do
# Questa Altera Starter FPGA Edition-64 vlog 2025.2 Compiler 2025.05 May 31 2025
```

```
# Start time: 01:27:20 on Oct 01,2026
# vlog -sv uart_tx.sv tb_uart_tx.sv
# ** Note: (vlog-220) 'C:/altera_lite/25.1std/questa_fse/win64/../modelsim.ini' is used as the ini file.
# -- Compiling module uart_tx
# -- Compiling module tb_uart_tx
#
# Top level modules:
#       tb_uart_tx
# End time: 01:27:21 on Oct 01,2026, Elapsed time: 0:00:01
# Errors: 0, Warnings: 0
# vsim work.tb_uart_tx
# Start time: 01:27:20 on Oct 01,2026
# ** Note: (vsim-3812) Design is being optimized...
# ** Note: (vopt-220) 'C:/altera_lite/25.1std/questa_fse/win64/../modelsim.ini' is used as the ini file.
# Loading sv_std.std
# Loading work.tb_uart_tx(fast)
# ERROR: no stop bit
# ERROR: expected 0x55, got 0xd5
# ERROR: no stop bit
# ERROR: expected 0x00, got 0xff
# ERROR: no stop bit
# ERROR: expected 0xff, got 0xb0
# ERROR: expected 0x41, got 0xfa
# ERROR: simulation timeout
# ** Note: $finish    : tb_uart_tx.sv(69)
#    Time: 1736 us  Iteration: 0  Instance: /tb_uart_tx
# End time: 01:27:22 on Oct 01,2026, Elapsed time: 0:00:02
# Errors: 0, Warnings: 0
```